

UAT Test Plan - Feature Remaining Features

Purpose

This guide is a stakeholder-facing user acceptance test plan for the recently delivered feature set that originally landed on the feature/remaining-features branch.

It covers step-by-step UI validation for:

- Login activity and session security
- Support ticket submission and admin ticket handling
- Mentions and in-app notifications
- Ride detail realtime updates
- Live session lifecycle and incident reporting

This document is the source of truth for the PDF artifact generated by the local export script.

Audience

- Product and stakeholder reviewers
- QA testers
- Engineering reviewers performing final regression validation

Out Of Scope

The following are intentionally excluded from this UAT plan:

- Real push notification delivery through FCM or APNs
- Real email delivery through SMTP or provider integrations
- Backend worker internals that have no visible UI outcome
- Unshipped live-session reliability hardening work

Required Test Environment

1. A running backend API and connected mobile client build.
2. At least two active app sessions or two devices for realtime validation.
3. Seeded or manually created ride, post, and support-ticket data.
4. For Scenarios 1â 4, the held-back features enabled on both sides and a build made with them: 'FEATURE_ACCOUNT_SECURITY' / 'EXPO_PUBLIC_FEATURE_ACCOUNT_SECURITY' (Scenario 1) and 'FEATURE_SUPPORT' / 'EXPO_PUBLIC_FEATURE_SUPPORT' (Scenarios 2â 4). Both are off in the closed beta.

Required Test Accounts

1. Standard rider A
2. Standard rider B
3. Standard rider C for optional multi-rider realtime checks
4. Admin rider: a row '(rider_id, 'admin')' in 'rider_roles', granted out of band. The rider must sign in again (or refresh) after the grant so the role reaches their token

Recommended Data Setup

1. Ensure at least one ride exists with a captain and one confirmed participant.
2. Ensure rider A can mention rider B by username.
3. Ensure the admin rider can access the same environment as the standard riders.
4. If possible, use fresh sessions before beginning the security scenarios.

Pass And Fail Guidance

Pass a scenario only when all expected UI outcomes occur without manual database fixes during execution.

Known issue (2026-09-28): the settings screen decides whether to show "Admin - Manage Tickets" from 'is_admin' on the rider profile, which is always 'false' since that column

was replaced by 'rider_roles'. Scenario 3 fails at step 4 until the client reads roles. The admin endpoints themselves check roles correctly.

Fail a scenario when:

- the screen does not open from the documented tap path
- the action succeeds but the expected state is not visible in the UI
- authorization or visibility rules are violated
- realtime updates require manual refresh when the feature is expected to update live

Log each failure with:

1. scenario identifier
2. account used
3. device or platform used
4. exact step number
5. actual result
6. screenshot or screen recording reference

Scenario 1 - Review Login Activity And Revoke A Session

Objective

Confirm that login activity and session inventory are visible, and that revoking a session blocks that session.

Primary Actor

Rider A on device 1 and device 2

Preconditions

1. Rider A can log in on two app sessions.

2. Both sessions point to the same backend environment.

Steps

1. Log in as rider A on device 1.
2. Log in as rider A on device 2.
3. On device 1, open My Profile.
4. Tap Settings.
5. Open Security.
6. Review the login activity list.
7. Review the active sessions list.
8. Revoke the session that represents device 2.
9. On device 2, attempt to refresh profile data or open another authenticated screen.

Expected Result

1. The login activity list shows recent login events.
2. The sessions list shows more than one active session before revocation.
3. The revoked session no longer remains valid on device 2.
4. Device 1 remains authenticated.

Scenario 2 - Submit A Support Ticket As A Rider

Objective

Confirm that a standard rider can create a support ticket and see it in personal ticket history.

Primary Actor

Rider B

Preconditions

1. Rider B is logged in.

Steps

1. Open My Profile.
2. Tap Settings.
3. In the Support section, tap Contact Support.
4. Select a ticket category.
5. Enter a subject.
6. Enter a detailed description.
7. Optionally add one or more attachment URLs.
8. Submit the ticket.
9. Review the ticket list shown in the Support Center.

Expected Result

1. The ticket is created successfully.
2. The rider sees the new ticket in their support history.
3. The ticket status appears with an initial open-style state.

Scenario 3 - Admin Reviews And Updates A Support Ticket

Objective

Confirm that an admin can open the ticket-management view, inspect a rider ticket, update status, and save an agent reply.

Primary Actor

Admin rider

Preconditions

1. Scenario 2 has created a ticket.
2. The admin account has the 'admin' role in 'rider_roles' and has signed in since it was granted.

Steps

1. Log in as the admin rider.
2. Open My Profile.
3. Tap Settings.
4. In the Support section, tap Admin - Manage Tickets.
5. Confirm the ticket list loads.
6. Open the ticket created by rider B.
7. Change the status to In Progress or Resolved.
8. Enter an agent reply.
9. Save the update.

Expected Result

1. The admin-only ticket list is accessible to the admin rider.
2. The selected ticket detail view shows rider information and ticket content.
3. The status update succeeds.
4. The saved agent reply appears in the ticket detail after update.

Scenario 4 - Non-Admin Cannot Access Admin Ticket Controls

Objective

Confirm that admin support controls are hidden from a non-admin rider.

Primary Actor

Standard rider C

Preconditions

1. Rider C is logged in.

Steps

1. Open My Profile.
2. Tap Settings.
3. Review the Support section.

Expected Result

1. Contact Support is visible.
2. Admin - Manage Tickets is not visible.

Scenario 5 - Create A Post With A Mention

Objective

Confirm that mentioning another rider in a post produces an in-app notification for the mentioned rider.

Primary Actor

Rider A mentioning rider B

Preconditions

1. Rider B has a known username.

2. Rider A is logged in.

Steps

1. Open the Feed.
2. Start creating a new post.
3. Enter content that includes the handle for rider B in the format @username.
4. Submit the post.
5. Log in as rider B or switch to rider B on another device.
6. Open Notifications.

Expected Result

1. The post is created successfully.
2. Rider B sees a new in-app notification for the mention.
3. The notification appears as unread initially.

Scenario 6 - Create A Comment With A Mention

Objective

Confirm that mentioning another rider inside a comment also creates a mention notification.

Primary Actor

Rider A mentioning rider B in comments

Preconditions

1. An existing post is available.
2. Rider A is logged in.

Steps

1. Open a post detail page.
2. Enter a new comment containing @username for rider B.
3. Submit the comment.
4. Switch to rider B.
5. Open Notifications.

Expected Result

1. The comment is created successfully.
2. Rider B receives a new mention notification for the comment.

Scenario 7 - Manage Read State In Notifications

Objective

Confirm that unread filtering, per-item mark-as-read, and mark-all behavior work in the Notifications center.

Primary Actor

Rider B

Preconditions

1. Scenarios 7 and 8 have produced at least one notification.

Steps

1. Open Notifications.
2. Confirm the unread count is visible near the top of the screen.
3. Switch to the Unread filter.
4. Tap one unread notification to mark it as read.
5. Confirm the list and unread count update.
6. Return to All.
7. Tap Mark all.

Expected Result

1. The unread filter shows only unread notifications.
2. Tapping an unread notification marks it read.
3. Mark all clears the remaining unread state.

Scenario 8 - Ride Detail Realtime Update For Ride Join Or Stop Event

Objective

Confirm that ride detail screens receive live updates through the lightweight ride-room channel.

Primary Actor

Captain on device 1 and participant on device 2

Preconditions

1. A ride exists that both users can view.
2. Both users are authenticated.
3. Both devices open the same ride detail screen.

Steps

1. On device 1, open the target ride detail screen and keep it visible.
2. On device 2, open the same ride detail screen.
3. Trigger a ride change that should broadcast to subscribers.
4. If the ride still allows join behavior, complete a join action from device 2.
5. If stop-management is available instead, request or update a stop from one device.
6. Observe the ride detail screen on the other device without manually refreshing.

Expected Result

1. The second device sees the ride update without requiring a full pull-to-refresh.
2. The broadcasted change matches the action that just occurred.
3. Unauthorized users should not be able to receive private ride updates.

Scenario 9 - Start And End A Live Session

Objective

Confirm that a ride captain can start and end a live session and that participants see the session state change.

Primary Actor

Ride captain and confirmed participant

Preconditions

1. A ride exists with a captain and a confirmed participant.
2. Both users can open the same ride detail screen.

Steps

1. On the captain device, open ride detail.
2. Start the live session.
3. On the participant device, remain on the same ride detail screen.
4. Confirm the participant sees live-session state information.
5. End the live session from the captain device.

Expected Result

1. Starting the session succeeds and reflects in the UI.
2. The participant sees the live session become active.
3. Ending the session updates both devices to show the ended state.

Scenario 10 - Report A Live Incident Or SOS

Objective

Confirm that a participant can report an incident and that other session participants see the event.

Primary Actor

Confirmed participant reporting to captain

Preconditions

1. Scenario 9 has an active live session.
2. Both participant and captain remain connected in the same ride.

Steps

1. On the participant device, trigger the SOS or incident action from ride detail.
2. Keep the captain device on the same ride detail screen.
3. Observe the live session UI on both devices.

Expected Result

1. The participant action succeeds without crashing the session UI.
2. The captain device receives visible incident state changes.
3. The participant remains in the live session after reporting.

Final Sign-Off Checklist

1. All scenarios that are in scope have a pass or fail outcome recorded.
2. Any failing scenario includes screenshots or notes for reproducibility.
3. Realtime flows were tested with two concurrent app sessions.
4. Unsupported provider integrations were not counted as UAT failures.

PDF Export Workflow

Use the local PDF generator script to create a PDF from this source file.

Command:

1. `'/opt/homebrew/bin/python3 scripts/generate_pdf.py docs/uat-test-plan-feature-remaining-features.md docs/uat-test-plan-feature-remaining-features.pdf'`

Notes:

1. The generator uses only the Python standard library.
2. Regenerate the PDF whenever this markdown file changes.
3. Treat this markdown file as the editable source of truth rather than the PDF.